

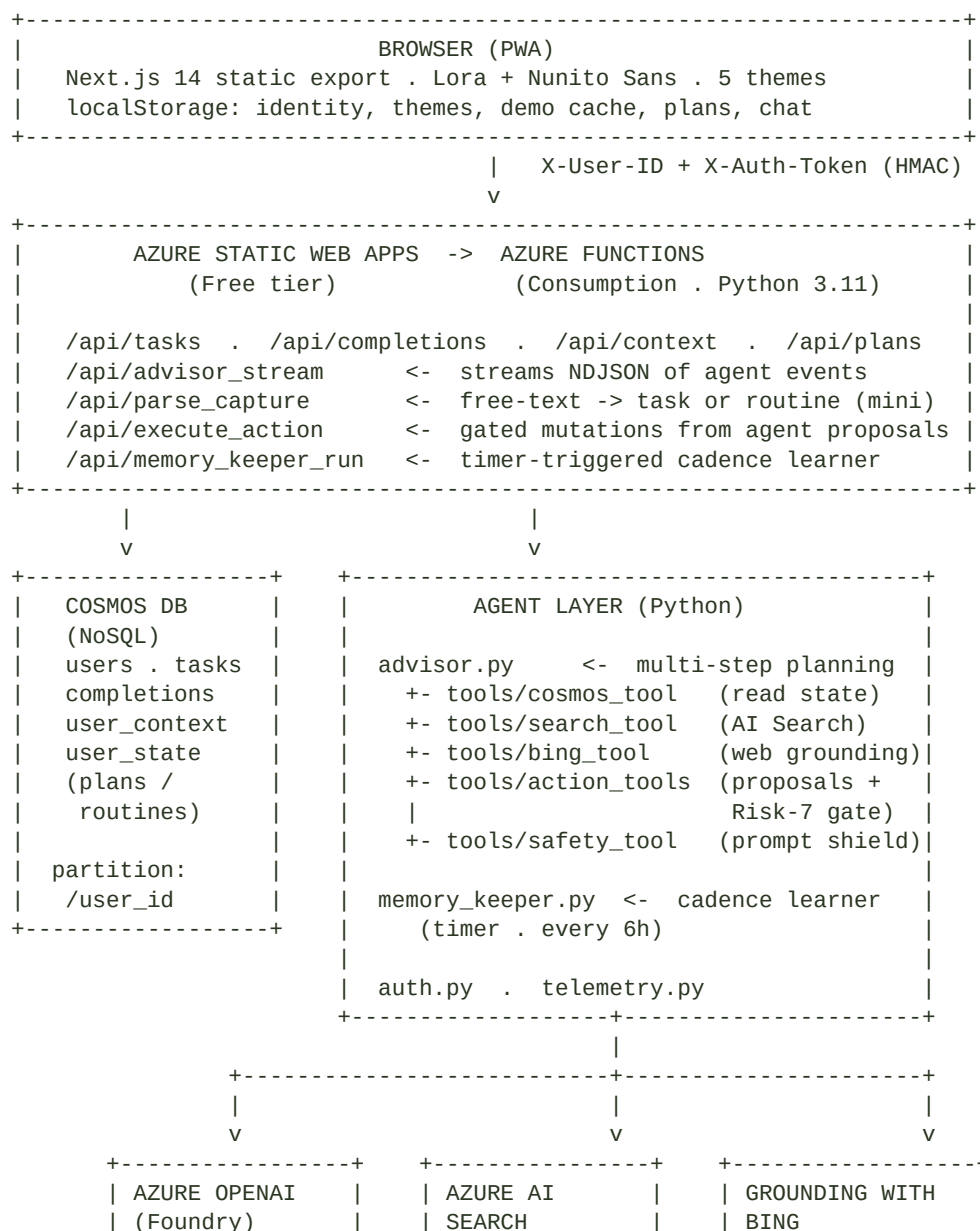
Heed — Solution Architecture

A gentle agentic personal assistant for what you forget. Built for the Microsoft CWB Hackathon 2026.

1. Problem & Solution

Most productivity apps treat people like project managers — lists, deadlines, guilt-tinged streaks. Real life isn't a project; it's a quiet stream of small things that quietly slip: utility bills, calls to family, savings goals, trip prep. Heed is an agentic personal assistant that learns the user's real cadences from behavior, surfaces what's at risk first, and adapts to actual capacity instead of an aspirational one. Every state-changing action is proposed by the agent and explicitly confirmed by the user — the agent proposes, the user disposes.

2. System Architecture



heed-advisor (gpt-5.4)	task_memory (vector+text)	Web facts for time-sensitive queries
heed-keeper (gpt-5.4-mini)	ph_calendar (cultural)	
heed-embed (text-embed-3)		
+-----+	+-----+	+-----+

Live demo: brave-pond-035757400.7.azurestaticapps.net · Source: github.com/anitohlm/Heed

3. Azure Services

Layer	Service	Role
Frontend	Azure Static Web Apps (Free tier)	Serves the pre-built Next.js 14 PWA at <i>web/out/</i> ; GitHub Actions auto-deploys on push to main.
Backend API	Azure Functions (Consumption, Python 3.11 v2)	All HTTP endpoints + the timer-triggered Memory Keeper.
Database	Azure Cosmos DB for NoSQL	<i>users</i> · <i>tasks</i> · <i>completions</i> · <i>user_context</i> · <i>user_state</i> . Every container partitioned on <i>/user_id</i> .
LLM	Azure OpenAI (AI Foundry)	<i>heed-advisor</i> (gpt-5.4) for the agentic loop; <i>heed-keeper</i> (gpt-5.4-mini) for capture, Memory Keeper, and Bing summarisation; <i>heed-embed</i> (text-embedding-3-small) for AI Search vectors.
Personal grounding	Azure AI Search	<i>task_memory</i> (vector + text over user tasks & notes) and <i>ph_calendar</i> (PH holidays + cultural events). Indexer pulls from the Cosmos change feed every 5 minutes.
Public grounding	Grounding with Bing	Called as a tool by the advisor for time-sensitive web facts; results cited with source URLs.
Content safety	Azure AI Content Safety	Prompt shield runs on every chat input before the advisor sees it (Risk-1 mitigation).
Secrets	Azure Key Vault + Managed Identity	No API keys in source. Functions read at runtime via MI; the frontend has zero secrets.
Observability	Application Insights	Spans around every advisor call, every tool invocation, every Cosmos write — debug an agent loop like a microservice.

4. Multi-Model Strategy

Three deployments in one *openai-heed* Foundry resource. The split keeps cost low on high-volume paths (capture parsing, the every-6-hours Memory Keeper) while the user-facing reasoning surface stays sharp. Eval and trade-offs documented in *docs/MULTI_MODEL_COMPARISON.md*.

Deployment	Model	Where it runs	Why
heed-advisor	gpt-5.4	advisor.py — the agentic loop	Multi-step planning, tool selection, weighing trade-offs across the user's full context.
heed-keeper	gpt-5.4-mini	parse_capture · memory_keeper · bing_tool	Narrow extraction + pattern detection. Sub-500ms latency on capture parsing.
heed-embed	text-embedding-3-small	search_tool — task_memory vectors	Semantic recall over task names + completion notes.

5. Grounding & the Agent Loop

The advisor reasons in two retrieval surfaces — one personal, one public — so it never confuses what's about *you* with what's about *the world*:

- **Personal data** via Azure AI Search. The *task_memory* index is a per-user vector + text store of tasks, completion history, and the last five completion notes per task; the *ph_calendar* index adds Philippine holidays and cultural events for local-time grounding.
- **Public web** via Grounding with Bing. The advisor calls *bing_search* for fresh facts ("is Memorial Day a long weekend this year?", "weather forecast for the trip?") and the response carries source URLs surfaced to the user.

On the action side, the advisor never writes to Cosmos directly. It calls *propose_action*; the frontend renders each proposal as a confirm chip; only after the user taps does */api/execute_action* route through *validate_action* (the Risk-7 multi-task gate) and hit Cosmos. The agent proposes; the user disposes. That separation is structural, not a behavioral hope.

6. Safety & Risk Model

Seven risk scenarios are mitigated in code. Full eval in *docs/SAFETY.md*.

Risk	Mitigation in code
1. Prompt injection via task names / chat	Azure AI Content Safety prompt shield on every chat input; advisor system prompt distrusts task content as instructions.
2. Hallucinated mutations ("I added that for you")	Every mutation goes through <i>propose_action</i> ; user must tap a confirm chip before <i>execute_action</i> fires.
3. Unsafe defer/skip on health-critical tasks	Tasks marked importance: 'non-negotiable' are excluded from auto-lighten.
4. Stale cadence learning	Memory Keeper requires ≥ 5 completions across ≥ 3 weeks before learning a cadence.
5. Cross-user data leak	Every Cosmos query partitioned on <i>/user_id</i> ; HMAC verifies X-User-ID matches X-Auth-Token bearer.

6. Fabricated URLs from web grounding	Bing tool wraps the official Grounding-with-Bing API; results carry real cited URLs surfaced to the user.
7. Multi-task destructive actions	validate_action gate blocks proposals with requires_confirmation=true until user_confirmed=true.

7. Compliance

No PII anywhere. The chelle persona, her Singapore trip, her overdue Maynilad bill, the missed calls to Mom — all synthetic. The demo Cosmos bucket and committed seeds contain no real data.

No keys in source. Secrets live in Azure Key Vault. Functions read at runtime via Managed Identity; the frontend has zero secrets and uses an HMAC token issued at registration. *.env.local* and *local.settings.json* are gitignored.